ACE ENGINEERING COLLEGE

ONE-WEEK TECHNICAL PROJECT REPORT

Project Title: Smart Campus Navigator – Campus Location Finder Mobile Application using Flutter

Submitted by:	Arige Navya
Roll Number(s):	24AG1A0503
Branch and Year:	B.Tech – Computer Science and Engineering, III Year
Academic Year:	2026–2027
Project Duration:	04/11/2026 to 10/11/2026 (7 Days)
Project Guide / Mentor:	Mr. P. Satish Bhasker
Organization / Training Institute:	ACE Engineering College, Hyderabad

Submitted to the Department of Computer Science and Engineering

TABLE OF CONTENTS

1. Certificate
2. Acknowledgement
3. Abstract
4. Introduction
5. Objectives of the Project
6. Problem Statement and Scope
7. Tools and Technologies Used
8. System Design / Methodology
9. Day-Wise Progress Report
10. Implementation and Results
11. Learning Outcomes
12. Challenges Faced and Solutions
13. Conclusion and Future Scope
14. References
15. Appendices / Supporting Evidence
16. Mentor Evaluation Sheet

1. CERTIFICATE

This is to certify that **Arige Navya**, bearing Roll Number **24AG1A0503**, of B.Tech – Computer Science and Engineering, III Year, has completed the one-week technical project entitled **"Smart Campus Navigator – Campus Location Finder Mobile Application using Flutter"** during 04/11/2026 to 10/11/2026 at ACE Engineering College, Hyderabad, under the guidance of Mr. P. Satish Bhasker.

Signature of Project Mentor: _____

Signature of Department Project Guide: _____

Date: _____ Place: Hyderabad

2. ACKNOWLEDGEMENT

I express my sincere gratitude to my project mentor Mr. P. Satish Bhasker for his valuable guidance, encouragement and timely suggestions throughout this one-week project. I thank the Head of the Department and all the faculty members of the Department of Computer Science and Engineering, ACE Engineering College, for providing the opportunity and the facilities to carry out this work.

I am also thankful to the Flutter and Dart open-source communities, whose documentation and examples helped me learn the framework quickly, and to my family and friends for their constant support.

Arige Navya
24AG1A0503

3. ABSTRACT

New students, parents and visitors often find it difficult to locate classrooms, laboratories, department offices and the library inside a large college campus. Asking people for directions wastes time and causes students to be late for classes and examinations. This one-week project presents **Smart Campus Navigator**, a simple Flutter and Dart mobile application that helps users search for any place on campus and reach it using a digital map and step-by-step walking directions. The app follows a three-layer design – presentation (screens and widgets), state (a ChangeNotifier controller) and data (place model, campus map data and services). Key features include search by name, room number or keyword, category filters (departments, classrooms, labs, libraries, offices, halls, facilities), an interactive offline campus map drawn with CustomPainter, place details with building, floor, room and timings, shortest-route calculation using Dijkstra's algorithm with distance and walking time, saved places, recently viewed places and a dark mode. Favourites and settings are stored on the device with shared_preferences. The

logic is covered by 10 unit and widget tests. The project gave hands-on experience in Flutter UI development, state management, graph algorithms, custom drawing and local persistence.

4. INTRODUCTION

Flutter is Google's open-source UI toolkit for building natively compiled applications for mobile, web and desktop from a single code base. Applications are written in Dart and the whole user interface is composed of widgets, which makes it fast to build attractive, consistent screens. Flutter's Material 3 widgets, hot reload and strong testing support make it a popular choice for both start-ups and large companies.

A college campus has many blocks, each with several floors, rooms and laboratories. Fresh students, guest faculty and visitors do not know where the Examination Cell, a particular lab or a classroom such as C-204 is located. Printed notice-board maps are hard to read and general map apps such as Google Maps do not show rooms inside a campus. A small campus-specific navigation app solves this problem.

The motivation for selecting this project was to learn Flutter app development end-to-end within one week – designing screens, managing state, drawing a custom map, applying a shortest-path algorithm and saving data on the device – while building something useful for every student of the college.

5. OBJECTIVES OF THE PROJECT

- To design and develop a cross-platform campus navigation mobile application using Flutter and Dart.
- To allow users to search classrooms, labs, departments, libraries and offices by name, room number or keyword, and filter them by category.
- To display an interactive digital map of the campus on which buildings can be tapped to see the places inside.
- To find the shortest walking route between two places and show the distance, walking time and step-by-step directions.
- To let users save favourite places and see recently viewed places, stored locally on the device.
- To verify the application with automated unit and widget tests.

6. PROBLEM STATEMENT AND SCOPE

Problem statement: Students and visitors waste time searching for classrooms, labs and offices on campus. There is no single place where the location, floor, room number and timings of every campus facility are available, and general map applications do not show the inside of a college campus. A simple, offline, campus-specific navigation app is needed.

Scope: Directory of campus places (name, category, building, floor, room, timings, description); search and category filter; interactive campus map with zoom and pan; shortest walking route

with distance, time and directions; place details; saved and recent places; light and dark themes; local storage on the device; automated tests.

Outside the scope: Live GPS tracking, indoor positioning (Wi-Fi/Bluetooth beacons), multi-floor indoor maps, an admin panel to edit places online, cloud synchronisation and publishing to the Play Store / App Store are outside the scope of this one-week project.

7. TOOLS AND TECHNOLOGIES USED

Tool / Technology	Purpose	Version / Details
Flutter SDK	Cross-platform UI framework used to build all screens	3.x (stable channel)
Dart	Programming language of the application	3.x
Material 3	Design system, widgets and light/dark themes	Built into Flutter
CustomPainter / InteractiveViewer	Drawing the offline campus map with zoom and pan	Built into Flutter
shared_preferences	Local storage of saved places, recent places and theme	^2.3.2 (pub.dev)
flutter_test	Unit and widget testing framework	Included with Flutter SDK
flutter_lints	Static analysis / coding standard rules	^5.0.0
Visual Studio Code	Code editor with Flutter and Dart extensions	Latest version
Google Chrome	Running the app (web target) and capturing screenshots	Stable
Git and GitHub	Version control and remote repository	Git 2.x, github.com
GitHub Actions	Continuous integration: analyze, test, build	ubuntu-latest runner

8. SYSTEM DESIGN / METHODOLOGY

Smart Campus Navigator is organised in three layers so that each part has a single responsibility (Figure 1).

Presentation layer: four tabs (Home, Map, Places, Saved) inside a bottom-navigation shell, plus the Place Detail and Directions screens. They are built from small reusable widgets such as PlaceTile and CampusMap (a CustomPainter inside an InteractiveViewer).

State layer: a CampusController extends ChangeNotifier. It provides search and category filtering, keeps the list of favourites and recently viewed places and the theme setting, and calls notifyListeners(); every screen rebuilds automatically through ListenableBuilder.

Data layer: the Place model and CampusData (road junctions, roads, buildings and 18 places), RouteService (Dijkstra's shortest path on the road graph and generation of directions) and StorageService (shared_preferences).

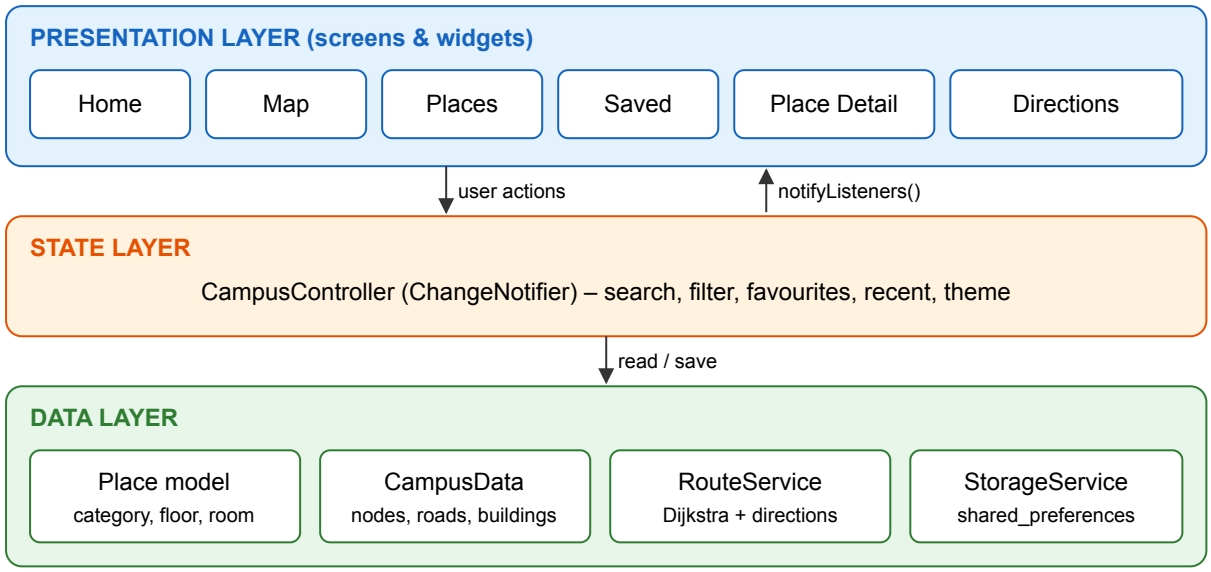


Figure 1: System architecture and data flow of Smart Campus Navigator

Route-finding method

The campus is modelled as a weighted graph. Road junctions are nodes with (x, y) map coordinates in metres, and each road segment is an edge whose weight is the straight-line distance between its two junctions. Every place stores the junction nearest to its building entrance. When the user chooses a start and a destination, Dijkstra's algorithm finds the shortest path between the two entrance junctions. The total distance is divided by an average walking speed of 1.2 m/s to estimate the walking time, and consecutive road segments in the same direction are merged into simple instructions such as “Walk 190 m north”.

Project folder structure

```
lib/
├── main.dart                 # loads saved data, starts the app, light/dark
theme
├── models/      place.dart   # Place, PlaceCategory, Building
├── data/        campus_data.dart        # junctions, roads, buildings, 18
places
├── services/    route_service.dart      # Dijkstra shortest path + directions
│                storage_service.dart    # shared_preferences
├── state/       campus_controller.dart  # ChangeNotifier
├── widgets/     campus_map.dart, place_tile.dart
└── screens/     home_shell, home, map, places, place_detail, navigate, saved
test/            route_service_test, search_test, widget_test
```

Methodology followed

- Requirement analysis – surveyed classmates about places that are hard to find and finalised the features and screens.
- Data collection – listed the campus blocks, floors, rooms and timings and sketched the road layout as a graph.
- Design – prepared the three-layer architecture, the data model and the screen flow.
- Incremental development – built models and campus data first, then services and state, then widgets and screens.
- Testing – wrote unit tests for route finding and search and a widget test for the home screen; fixed the issues found.
- Execution and documentation – ran the app, captured screenshots, wrote the README and this report.

9. DAY-WISE PROGRESS REPORT

Day	Activities Performed	Deliverables / Outcomes
Day 1 04/11/2026	Orientation, topic selection (campus navigator) and requirement analysis; listed features, screens and place details to be stored	Objectives, feature list and scope finalised
Day 2 05/11/2026	Studied Flutter widgets, Material 3, CustomPainter and state management; set up Flutter SDK and VS Code; created the project and Git repository	Development environment ready, project scaffold committed
Day 3 06/11/2026	Collected campus data (blocks, floors, rooms, timings); designed the road graph, Place model and three-layer architecture	Architecture diagram, models and campus data
Day 4 07/11/2026	Implemented RouteService (Dijkstra), StorageService, CampusController and the CustomPainter campus map	Core modules working; shortest route drawn on the map
Day 5 08/11/2026	Built Home, Places (search and filters), Place Detail, Directions and Saved screens; integrated the bottom navigation; wrote unit and widget tests	All features integrated, tests written
Day 6 09/11/2026	Ran the app, captured screenshots, fixed UI issues, added dark mode, wrote README and GitHub Actions CI	Results recorded, documentation ready
Day 7 10/11/2026	Final demonstration to the mentor, pushed the code to GitHub, prepared and submitted this report	Project completed and presented

10. IMPLEMENTATION AND RESULTS

The application was implemented module by module in Dart using Flutter. The table below lists the main modules, followed by screenshots of the application and the test cases.

Module	File(s)	Description
Data model	models/place.dart	Place with category, building, floor, room, timings and tags; 7 categories with icon and colour
Campus data	data/campus_data.dart	12 road junctions, 13 road segments, 9 buildings and 18 places
Route finding	services/route_service.dart	Dijkstra shortest path, distance, walking time, step-by-step directions
Storage	services/storage_service.dart	Saves favourites, recent places and dark mode with shared_preferences
State / logic	state/campus_controller.dart	Search, category filter, favourites, last-5 recent places, theme
Campus map	widgets/campus_map.dart	CustomPainter map inside InteractiveViewer (zoom/pan), building highlight, route line, tap detection
Home	screens/home_screen.dart	Gradient header, search, Get Directions, category grid, recent and popular places
Places	screens/places_screen.dart	Live search and category chips with result count
Place detail	screens/place_detail_screen.dart	Map with highlighted building, details, bookmark, Navigate here
Directions	screens/navigate_screen.dart	From/To pickers, distance, time, floor, route on map, step list
Map / Saved	screens/map_screen.dart, saved_screen.dart	Tap a building for a bottom sheet of its places; saved places list

Testing

Automated tests were written with flutter_test. Unit tests check the route-finding and search logic and a widget test checks that the home screen is built correctly.

Test case	Type	Expected result	Status
Shortest path Main Gate → CSE Block	Unit	m0 → m1 → m2 → r2	Pass
Route distance and walking time	Unit	290 m, 4 min	Pass
Directions merge straight segments	Unit	“Walk 190 m north”, “Walk 100 m east”	Pass
Same building gives zero distance	Unit	0 m	Pass

Every place reachable from the gate	Unit	Non-empty path for all 18 places	Pass
Search by name, room and tag	Unit	Only matching places	Pass
Filter by category (Labs)	Unit	4 labs	Pass
Favourites saved and reloaded	Unit	Same data after restart	Pass
Recent list keeps latest 5	Unit	5 items, newest first	Pass
Home screen shows title and categories	Widget	Title, Labs, Get Directions visible	Pass

Screenshots of the application

The application was opened in a phone-sized window (412 × 892). The following screenshots show the main screens.

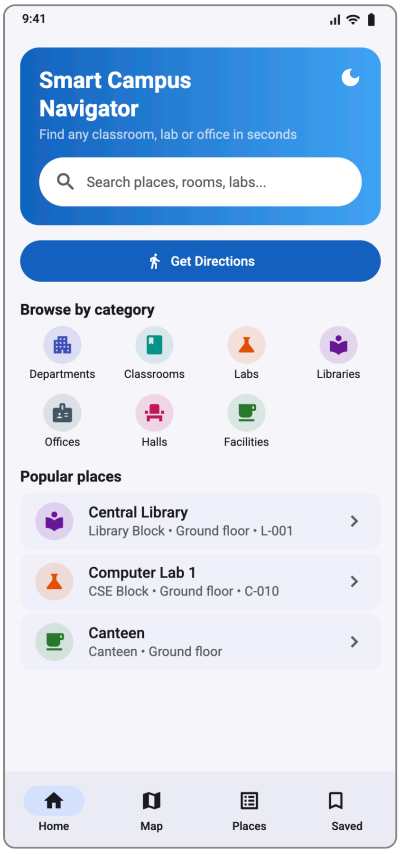


Figure 2: Home screen

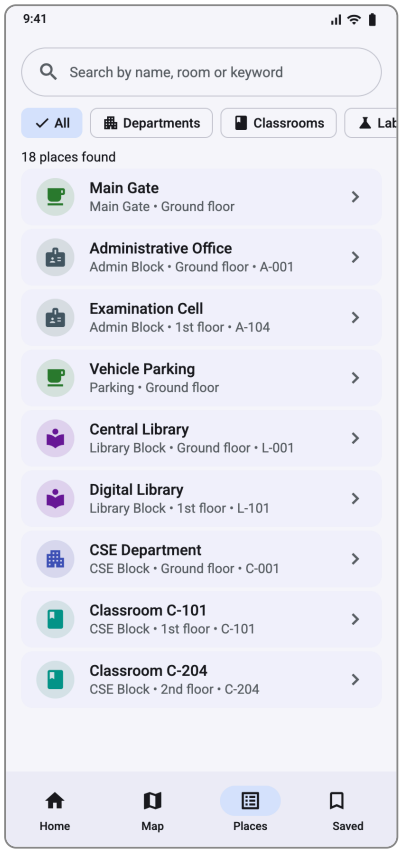


Figure 3: All campus places

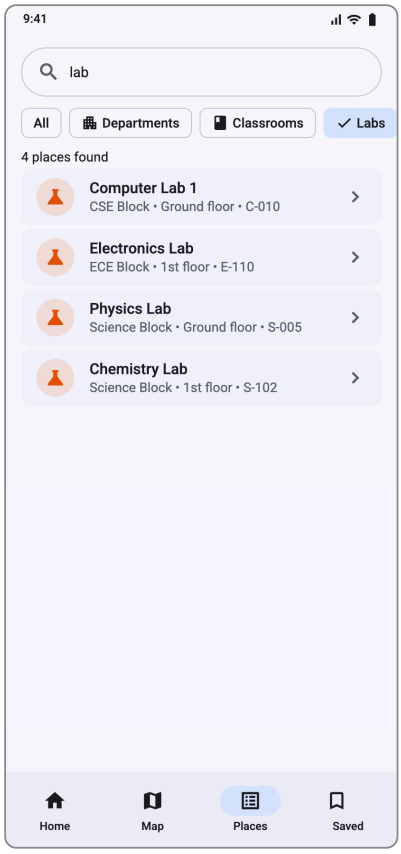


Figure 4: Searching "lab" with Labs filter

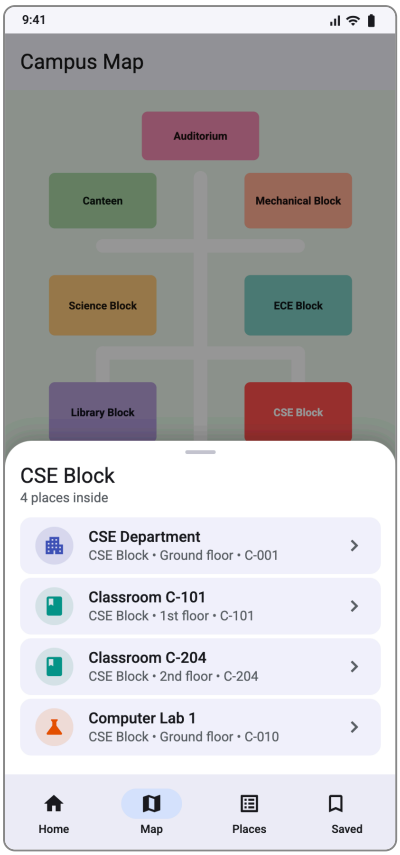


Figure 5: Campus map – tapping CSE Block

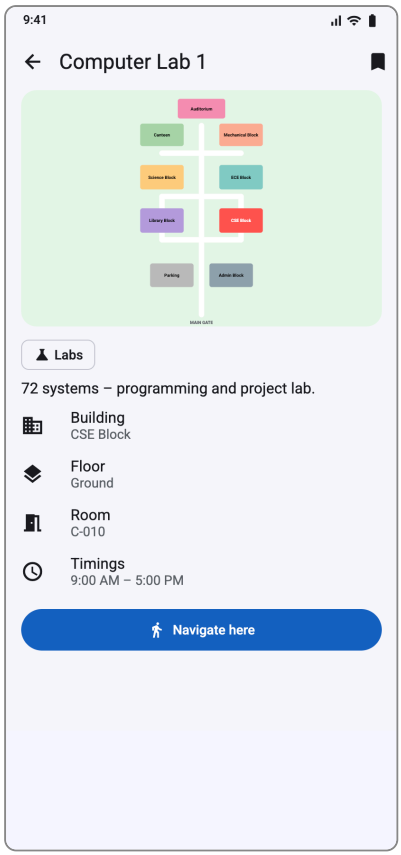


Figure 6: Place details



Figure 7: Route Main Gate → Computer Lab 1



Figure 8: Route Canteen → Examination Cell

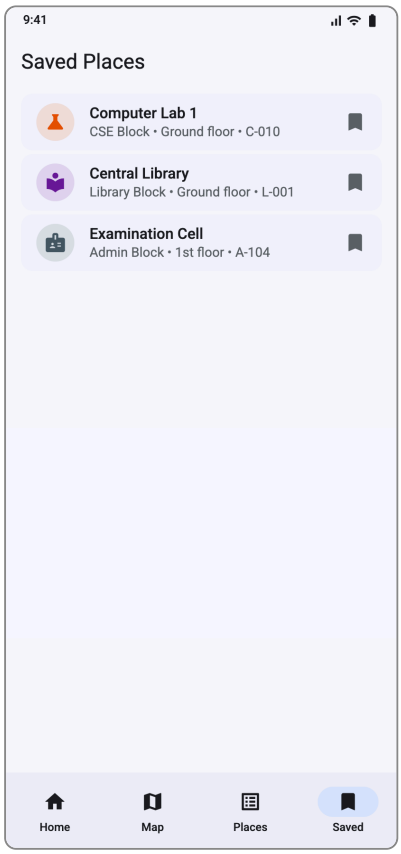


Figure 9: Saved places

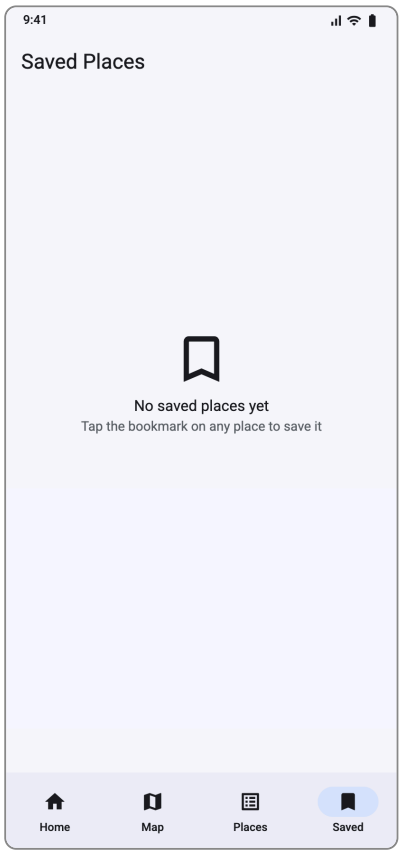


Figure 10: Saved tab – empty state

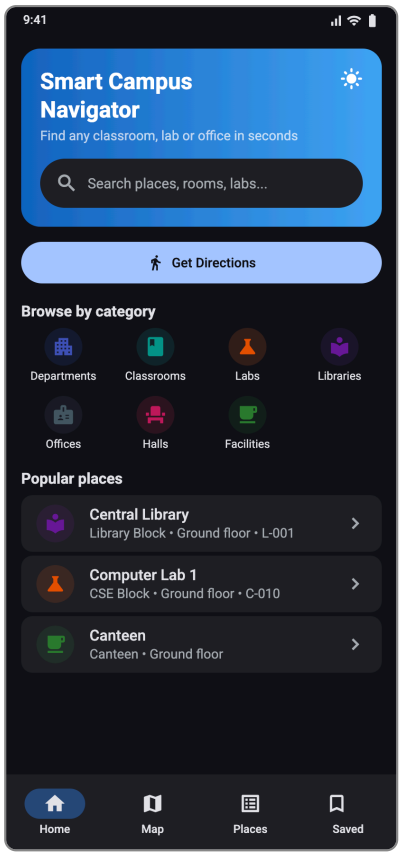


Figure 11: Dark mode

Result / Observation: All planned features were implemented. Searching “lab” with the Labs filter lists the 4 laboratories with their block, floor and room. Selecting Main Gate → Computer Lab 1 draws the shortest route on the map and shows 290 m and about 4 minutes of walking with three simple directions; Canteen → Examination Cell gives 410 m and about 6 minutes. Saved places and the theme remain after the app is restarted.

11. LEARNING OUTCOMES

- Building user interfaces from Flutter widgets and Material 3 components (Scaffold, NavigationBar, Card, ChoiceChip, DropdownButtonFormField, bottom sheets).
- Managing application state with ChangeNotifier and ListenableBuilder, and keeping several screens in sync.
- Drawing a custom zoomable map with CustomPainter and InteractiveViewer and detecting taps on buildings.
- Applying Dijkstra's shortest-path algorithm to a real problem and turning the path into human-readable directions.
- Saving data on the device with shared_preferences.
- Writing unit and widget tests and using the analyzer to keep code quality high; using Git, GitHub and GitHub Actions.

12. CHALLENGES FACED AND SOLUTIONS

Challenge	Action / Solution	Outcome
Google Maps does not show buildings and rooms inside the campus	Drew an offline campus map with CustomPainter from our own coordinates	Works without internet or API key
Finding the shortest route between any two places	Modelled roads as a graph and used Dijkstra's algorithm	Correct shortest route every time
Too many small directions (one per segment)	Merged consecutive segments in the same direction	Short, clear 3–4 step directions
Several rooms in one building	Each place stores its building, floor and room; map highlights the building	User sees block, floor and room
Tapping the map after zooming	Converted tap position back to map coordinates using the scale factor	Correct building detected
Keeping all tabs updated after saving a place	One ChangeNotifier controller for all screens	Screens refresh instantly

13. CONCLUSION AND FUTURE SCOPE

Conclusion: The project achieved all of its objectives. Smart Campus Navigator lets students and visitors find any classroom, lab, department, library or office in a few taps, see it on a digital campus map and follow the shortest walking route with clear directions. The app was developed in one week using Flutter and Dart with a clean three-layer design and verified with automated tests.

Future scope:

- Live GPS location of the user on the campus map.
- Indoor, floor-by-floor maps and indoor positioning with Wi-Fi or Bluetooth beacons.
- Admin panel and Firebase backend so that places and timings can be updated without a new app release.
- QR codes at building entrances to set the starting point instantly.
- Class timetable integration (“take me to my next class”) and event notifications.
- Voice search, accessibility routes (ramps and lifts) and multi-language support.

14. REFERENCES

[1] Flutter Team, Google – Flutter documentation, https://docs.flutter.dev (accessed November 2026).
[2] Dart Team, Google – Dart language tour, https://dart.dev/language (accessed November 2026).
[3] Google – Material Design 3 guidelines, https://m3.material.io (accessed November 2026).

[4] Flutter API – CustomPainter class, https://api.flutter.dev/flutter/rendering/CustomPainter-class.html.
[5] Flutter Team – shared_preferences package, https://pub.dev/packages/shared_preferences.
[6] Flutter Team – Testing Flutter apps, https://docs.flutter.dev/testing/overview.
[7] T. H. Cormen et al., Introduction to Algorithms, 3rd ed., MIT Press, 2009 – Chapter 24 (Dijkstra's algorithm).
[8] M. L. Napoli, Beginning Flutter: A Hands-On Guide to App Development, Wiley, 2019.

15. APPENDICES / SUPPORTING EVIDENCE

A. Source code repository

https://github.com/navyaarige098-del/Smart-Campus-Navigator – contains the complete source code, README with screenshots and a GitHub Actions workflow that runs the analyzer, tests and a web build on every push.

B. Code extract – place model (lib/models/place.dart)

```
class Place {
  const Place({required this.id, required this.name, required this.category,
      required this.building, required this.floor, required this.entranceNode,
      this.room = '', this.timings = '9:00 AM – 5:00 PM',
      this.description = '', this.tags = const []});

  final String id, name, building, floor, room, timings, description;
  final PlaceCategory category;
  final List<String> tags;
  final String entranceNode; // road junction nearest to the entrance

  bool matches(String query) { /* name, room, category or tag */ }
}
```

C. Code extract – shortest path (lib/services/route_service.dart)

```
List<String> shortestPath(String from, String to) {
  final dist = {for (final n in _nodes.keys) n: double.infinity};
  final prev = <String, String>{};
  final open = {..._nodes.keys};
  dist[from] = 0;
  while (open.isNotEmpty) {
    final u = open.reduce((a, b) => dist[a]! <= dist[b]! ? a : b);
    if (dist[u] == double.infinity || u == to) break;
    open.remove(u);
    for (final e in (_adj[u] ?? {}).entries) {
      final alt = dist[u]! + e.value;
      if (alt < dist[e.key]!) { dist[e.key] = alt; prev[e.key] = u; }
    }
  }
  // walk back through prev[] to build the path
}
```

D. Commands used to run and test the application

```
flutter create . --platforms=android,ios,web
flutter pub get
flutter analyze
flutter test
flutter run -d chrome          # or an Android / iOS device
```

16. MENTOR EVALUATION SHEET

Student Name(s): Arige Navya (24AG1A0503)
Project Title: Smart Campus Navigator – Campus Location Finder Mobile Application using Flutter
Mentor Name: Mr. P. Satish Bhasker

Evaluation Criterion	Maximum Marks	Marks Awarded
Understanding of problem and objectives	10	
Technical knowledge and methodology	20	
Implementation / practical work	25	
Results and analysis	15	
Report quality and documentation	15	
Presentation and viva-voce	15	
Total	**100**	

Mentor Remarks: _____

_____

Mentor Signature: _____ Date: _____

HoD SIGNATURE